

R-Control IP4 – Modbus TCP Schnittstelle

Diese Dokumentation beschreibt die **Modbus-TCP-Slave**-Schnittstelle des R-Control IP4 aus Integrationssicht.

Stand: Implementierung in `components/modbus_server` (Firmware-Repo). Das Mapping ist auf maximal **4 Relais** und **4 Eingänge** ausgelegt.

1. Überblick

Zweck

- Relais über Coils schalten.
- Eingangs-/Relaisstatus sowie NTC-Telemetrie über Discrete Inputs und Input Registers lesen.

Eigenschaften

- Protokoll: Modbus TCP (Slave)
- Default Port: `502`
- Unit-ID: konfigurierbar (Default `1`)
- Netzwerk-Binding: bindet, wenn verfügbar, an das Ethernet-Netif `ETH_DEF` (ansonsten ohne explizites Netif-Binding).

2. Aktivierung & Konfiguration

Die Einstellungen liegen in der Konfiguration unter:

- `config.interfaces.modbus_tcp.enabled` (bool)
- `config.interfaces.modbus_tcp.port` (int, `1..65535`, Default `502`)
- `config.interfaces.modbus_tcp.unit_id` (int, `1..247`, Default `1`)

Im Web-UI: **Interfaces** → **Modbus TCP Einstellungen**.

3. Protokollparameter & Limits

Datenbereiche (Firmware-intern)

- Coils: 8 Bits (davon sind `0..3` belegt; `4..7` derzeit unbenutzt)
- Discrete Inputs: 16 Bits
- Holding Registers: 16 Register (aktuell reserviert)
- Input Registers: 80 Register

Aktualisierung / Zyklus

- Coil-Schreibzugriffe werden zyklisch auf die Relais angewendet (ca. alle 100 ms).

4. Adressierung & Datentypen

Wichtige Hinweise für Modbus-Clients:

- In dieser Dokumentation sind Adressen als **0-basierte Offsets** angegeben.
- Viele Tools (SCADA/HMI) zeigen **1-basierte** Adressen an. Dann gilt oft: Offset **N** → Anzeige **N+1**.
- Register-Endianess:
 - Modbus ist pro 16-bit Register big-endian.
 - Für 32-bit Werte nutzt die Firmware **Low-Word zuerst, High-Word danach**:
 - `value32 = lo16 | (hi16 << 16)`
- Signed/Unsigned:
 - Temperatur liegt in einem 16-bit Register als **int16 (°C×100)**. Clients müssen das Register als signed interpretieren.

5. Register Map

Hinweis: Alle Adressen/Offsets sind **0-basiert**.

5.1 Überblick (alle Bereiche)

Bereich	FC	Offset / Range	Typ	Zugriff	Bedeutung
Coils	01/05/15	0..3	bit	R/W	Relais (R1..R4) gewünschter Zustand (wird zyklisch angewendet)
Discrete Inputs	02	0..3	bit	R	Eingänge (S1..S4) pressed
Discrete Inputs	02	4..7	bit	R	Relais (R1..R4) Ist-Zustand
Discrete Inputs	02	8..11	bit	R	Eingänge (S1..S4) detached
Discrete Inputs	02	12..15	bit	R	Eingänge (S1..S4) enabled
Input Registers	04	0	int16 ×0.01	R	Temperatur °C×100
Input Registers	04	1	uint16	R	NTC Status (<code>ntc_sensor_status_t</code>)
Input Registers	04	2	uint16	R	NTC mV
Input Registers	04	3	uint16	R	NTC raw ADC
Input Registers	04	10..13	uint16	R	pressed (S1..S4) als 0/1
Input Registers	04	20..23	uint16	R	last event id (S1..S4, <code>BUTTON_EVENT_*</code>)

Bereich	FC	Offset / Range	Typ	Zugriff	Bedeutung
Input Registers	04	30..37	uint16	R	event counter (S1..S4) je 2 Register: lo/hi (32-bit)
Input Registers	04	40..47	uint16	R	last timestamp ms (S1..S4) je 2 Register: lo/hi (32-bit)
Input Registers	04	50..53	uint16	R	input enabled (S1..S4) als 0/1
Input Registers	04	54..57	uint16	R	input detached (S1..S4) als 0/1
Input Registers	04	60..63	uint16	R	relay state (R1..R4) als 0/1
Holding Registers	03/06/16	0..15	uint16	R/W	reserviert (aktuell ohne Semantik)

Coils (FC 01/05/15) – Relais schalten

Start Offset: 0

Offset	Bedeutung	Zugriff
0	Relay 0 (R1) gewünschter Zustand (0/1)	R/W
1	Relay 1 (R2) gewünschter Zustand (0/1)	R/W
2	Relay 2 (R3) gewünschter Zustand (0/1)	R/W
3	Relay 3 (R4) gewünschter Zustand (0/1)	R/W

Verhalten: Coil-Schreibzugriffe werden zyklisch (ca. alle 100 ms) auf die Relais angewendet.

Discrete Inputs (FC 02) – Status-Bits (read-only)

Start Offset: 0

Bit	Bedeutung
0..3	Input pressed (S1..S4): 1 = gedrückt
4..7	Relay state (R1..R4): 1 = Relais ist AN
8..11	Input detached (S1..S4): 1 = entkoppelt
12..15	Input enabled (S1..S4): 1 = Kanal aktiv

Input Registers (FC 04) – Telemetrie & Event-Infos (read-only)

Start Offset: 0

Temperatur / NTC Snapshot

Offset	Typ	Bedeutung
0	int16 (x100)	Temperatur in °C × 100 (z. B. 2345 = 23.45 °C). Bei Fehlerstatus kann der Wert 0 sein.
1	uint16	NTC Status (<code>ntc_sensor_status_t</code>).
2	uint16	NTC Messwert in mV (ungeklemmt auf 0..65535).
3	uint16	NTC Raw ADC (ungeklemmt auf 0..65535).

Eingänge: pressed / last event / counter / timestamp

Für jeden Eingang *i* (0..3):

Offset	Typ	Bedeutung
10 + i	uint16	pressed: 1 = gedrückt, 0 = nicht gedrückt
20 + i	uint16	last event id (entspricht <code>BUTTON_EVENT_*</code> ID)
30 + 2*i	uint16	event counter low-word
30 + 2*i + 1	uint16	event counter high-word
40 + 2*i	uint16	last timestamp low-word (ms)
40 + 2*i + 1	uint16	last timestamp high-word (ms)

32-bit Rekonstruktion:

- `event_count_32 = cnt_lo | (cnt_hi << 16)`
- `timestamp_ms_32 = ts_lo | (ts_hi << 16)`

Konfig-/Status-Spiegel

Offset	Typ	Bedeutung
50 + i	uint16	input enabled (S1..S4): 1/0
54 + i	uint16	input detached (S1..S4): 1/0
60 + i	uint16	relay state (R1..R4): 1/0

Holding Registers (FC 03/06/16)

Aktuell reserviert, aber in der Firmware noch ohne definierte Semantik.

6. Hinweise zur Integration (Best Practices)

- **Relais-Istzustand** wird redundant bereitgestellt:
 - Discrete Inputs Bits 4..7

- Input Registers **60..63** (je 0/1)
- **Coils** sind der Schreibkanal; die Firmware wendet Coil-Zustände zyklisch auf die realen Relais an (ca. 100 ms).
- **Toggle vermeiden**: Schreiben von Coils ist idempotent (0/1). Für deterministische Steuerungen sollten Integratoren in der Regel **0/1** statt "toggle"-Logik verwenden.
- **"detached"** bedeutet: Eingang ist von der Relais-Kopplung entkoppelt; Events/Status existieren weiterhin.

7. Security-Hinweise

Modbus TCP ist in dieser Firmware **nicht authentifiziert**.

Empfohlene Maßnahmen:

- Nur im vertrauenswürdigen LAN/OT-Netz betreiben.
- Zugriff per Firewall/VLAN auf definierte Quell-IPs begrenzen.
- Falls ein Modbus-Gateway/Proxy genutzt wird: dort Authentifizierung/ACLs/TLS erzwingen.

8. Beispiele

8.1 Beispiel: Temperatur lesen

- FC04, Offset **0**, Länge **2** (Temperatur + NTC-Status)

8.2 Beispiel: Relais 1 einschalten

- FC05 (Write Single Coil), Offset **0**, Value **1**

8.3 Beispiel: Relaiszustand lesen

- FC02 (Read Discrete Inputs), Offset **4**, Länge **4** (Bits 4..7)

9. Troubleshooting

- **Keine Verbindung**: Port/Firewall prüfen (`config.interfaces.modbus_tcp.port`, Default **502**).
- **Falsche Offsets**: Prüfen, ob der Client 0- oder 1-basiert adressiert.
- **Unit-ID passt nicht**: `config.interfaces.modbus_tcp.unit_id` prüfen (1..247).
- **Temperatur unplausibel**: Register als `int16` (signed) interpretieren und Skalierung $\times 0.01$ anwenden.

10. Home Assistant Beispiel (configuration.yaml)

Die folgende Beispielkonfiguration bindet R-Control IP4 via **Modbus TCP** ein und erstellt:

- 4 **Coil-Switches** (Modbus Coils) als "Schreibkanal" für die Relais
- 4 **Relay-State Binary-Sensoren** (FC02 Bits 4..7) als "Istzustand" der Relais
- 4 **Template-Switches (syncd)**, die den Istzustand anzeigen, aber über Coils schalten
- Temperatur/NTC-Messwerte als `sensor` (Input Registers)
- Eingangsstatus (pressed/enabled/detached) als `binary_sensor`
- Button-Event-ID, Counter und Timestamp als `sensor` + Template-Kombination (2×16-bit → 32-bit)

Wichtig:

- Die hier dokumentierten Offsets sind **0-basiert**.
- Home Assistant nutzt beim Modbus-Integration-Backend typischerweise ebenfalls **0-basierte** Registeradressen. Falls du im UI "um 1 verschoben" siehst, prüfe die Doku deines verwendeten Clients/Frontends.

```
modbus:
- name: rcontrol_ip4
  type: tcp
  host: 192.168.1.50 # <-- IP vom Gerät
  port: 502
  # Optional: Verzögerung zwischen Requests
  # delay: 0

# Relais über Coils (FC01/05) – reiner Schreibkanal
# (Istzustand kommt separat über Discrete Inputs Bits 4..7)
switches:
- name: "rcontrol_relay1_coil"
  slave: 1
  address: 0
  write_type: coil
- name: "rcontrol_relay2_coil"
  slave: 1
  address: 1
  write_type: coil
- name: "rcontrol_relay3_coil"
  slave: 1
  address: 2
  write_type: coil
- name: "rcontrol_relay4_coil"
  slave: 1
  address: 3
  write_type: coil

# Discrete Inputs (FC02)
binary_sensors:
  # Eingänge pressed (Bits 0..3)
- name: "rcontrol_s1_pressed"
  slave: 1
  address: 0
  input_type: discrete_input
- name: "rcontrol_s2_pressed"
  slave: 1
  address: 1
  input_type: discrete_input
- name: "rcontrol_s3_pressed"
  slave: 1
  address: 2
  input_type: discrete_input
- name: "rcontrol_s4_pressed"
  slave: 1
  address: 3
  input_type: discrete_input
```

```
# Relais-Istzustand (Bits 4..7)
- name: "rcontrol_relay1_state"
  slave: 1
  address: 4
  input_type: discrete_input
- name: "rcontrol_relay2_state"
  slave: 1
  address: 5
  input_type: discrete_input
- name: "rcontrol_relay3_state"
  slave: 1
  address: 6
  input_type: discrete_input
- name: "rcontrol_relay4_state"
  slave: 1
  address: 7
  input_type: discrete_input

# detached (Bits 8..11)
- name: "rcontrol_s1_detached"
  slave: 1
  address: 8
  input_type: discrete_input
- name: "rcontrol_s2_detached"
  slave: 1
  address: 9
  input_type: discrete_input
- name: "rcontrol_s3_detached"
  slave: 1
  address: 10
  input_type: discrete_input
- name: "rcontrol_s4_detached"
  slave: 1
  address: 11
  input_type: discrete_input

# enabled (Bits 12..15)
- name: "rcontrol_s1_enabled"
  slave: 1
  address: 12
  input_type: discrete_input
- name: "rcontrol_s2_enabled"
  slave: 1
  address: 13
  input_type: discrete_input
- name: "rcontrol_s3_enabled"
  slave: 1
  address: 14
  input_type: discrete_input
- name: "rcontrol_s4_enabled"
  slave: 1
  address: 15
  input_type: discrete_input
```

```
# Input Registers (FC04)
sensors:
- name: "rcontrol_ntc_temperature"
  slave: 1
  address: 0
  input_type: input
  data_type: int16
  scale: 0.01
  precision: 2
  unit_of_measurement: "°C"
  device_class: temperature

- name: "rcontrol_ntc_status"
  slave: 1
  address: 1
  input_type: input
  data_type: uint16

- name: "rcontrol_ntc_mv"
  slave: 1
  address: 2
  input_type: input
  data_type: uint16
  unit_of_measurement: "mV"

- name: "rcontrol_ntc_raw_adc"
  slave: 1
  address: 3
  input_type: input
  data_type: uint16

# last event id (20..23)
- name: "rcontrol_s1_last_event_id"
  slave: 1
  address: 20
  input_type: input
  data_type: uint16
- name: "rcontrol_s2_last_event_id"
  slave: 1
  address: 21
  input_type: input
  data_type: uint16
- name: "rcontrol_s3_last_event_id"
  slave: 1
  address: 22
  input_type: input
  data_type: uint16
- name: "rcontrol_s4_last_event_id"
  slave: 1
  address: 23
  input_type: input
  data_type: uint16
```



```
# Event Counter (low/high) und Timestamp (low/high)
# S1
- name: "rcontrol_s1_event_cnt_lo"
  slave: 1
  address: 30
  input_type: input
  data_type: uint16
- name: "rcontrol_s1_event_cnt_hi"
  slave: 1
  address: 31
  input_type: input
  data_type: uint16
- name: "rcontrol_s1_ts_ms_lo"
  slave: 1
  address: 40
  input_type: input
  data_type: uint16
- name: "rcontrol_s1_ts_ms_hi"
  slave: 1
  address: 41
  input_type: input
  data_type: uint16

# S2
- name: "rcontrol_s2_event_cnt_lo"
  slave: 1
  address: 32
  input_type: input
  data_type: uint16
- name: "rcontrol_s2_event_cnt_hi"
  slave: 1
  address: 33
  input_type: input
  data_type: uint16
- name: "rcontrol_s2_ts_ms_lo"
  slave: 1
  address: 42
  input_type: input
  data_type: uint16
- name: "rcontrol_s2_ts_ms_hi"
  slave: 1
  address: 43
  input_type: input
  data_type: uint16

# S3
- name: "rcontrol_s3_event_cnt_lo"
  slave: 1
  address: 34
  input_type: input
  data_type: uint16
- name: "rcontrol_s3_event_cnt_hi"
  slave: 1
  address: 35
```

```

    input_type: input
    data_type: uint16
  - name: "rcontrol_s3_ts_ms_lo"
    slave: 1
    address: 44
    input_type: input
    data_type: uint16
  - name: "rcontrol_s3_ts_ms_hi"
    slave: 1
    address: 45
    input_type: input
    data_type: uint16

# S4
  - name: "rcontrol_s4_event_cnt_lo"
    slave: 1
    address: 36
    input_type: input
    data_type: uint16
  - name: "rcontrol_s4_event_cnt_hi"
    slave: 1
    address: 37
    input_type: input
    data_type: uint16
  - name: "rcontrol_s4_ts_ms_lo"
    slave: 1
    address: 46
    input_type: input
    data_type: uint16
  - name: "rcontrol_s4_ts_ms_hi"
    slave: 1
    address: 47
    input_type: input
    data_type: uint16

```

template:

```

  - sensor:
    - name: "rcontrol_s1_event_count"
      state: >-
        {{ (states('sensor.rcontrol_s1_event_cnt_hi')|int(0) * 65536) +
        (states('sensor.rcontrol_s1_event_cnt_lo')|int(0)) }}
      icon: mdi:counter
    - name: "rcontrol_s1_last_timestamp_ms"
      state: >-
        {{ (states('sensor.rcontrol_s1_ts_ms_hi')|int(0) * 65536) +
        (states('sensor.rcontrol_s1_ts_ms_lo')|int(0)) }}
      icon: mdi:clock-outline

    - name: "rcontrol_s2_event_count"
      state: >-
        {{ (states('sensor.rcontrol_s2_event_cnt_hi')|int(0) * 65536) +
        (states('sensor.rcontrol_s2_event_cnt_lo')|int(0)) }}
      icon: mdi:counter
    - name: "rcontrol_s2_last_timestamp_ms"

```

```

state: >-
  {{ (states('sensor.rcontrol_s2_ts_ms_hi')|int(0) * 65536) +
(states('sensor.rcontrol_s2_ts_ms_lo')|int(0)) }}
  icon: mdi:clock-outline

- name: "rcontrol_s3_event_count"
  state: >-
    {{ (states('sensor.rcontrol_s3_event_cnt_hi')|int(0) * 65536) +
(states('sensor.rcontrol_s3_event_cnt_lo')|int(0)) }}
    icon: mdi:counter
- name: "rcontrol_s3_last_timestamp_ms"
  state: >-
    {{ (states('sensor.rcontrol_s3_ts_ms_hi')|int(0) * 65536) +
(states('sensor.rcontrol_s3_ts_ms_lo')|int(0)) }}
    icon: mdi:clock-outline

- name: "rcontrol_s4_event_count"
  state: >-
    {{ (states('sensor.rcontrol_s4_event_cnt_hi')|int(0) * 65536) +
(states('sensor.rcontrol_s4_event_cnt_lo')|int(0)) }}
    icon: mdi:counter
- name: "rcontrol_s4_last_timestamp_ms"
  state: >-
    {{ (states('sensor.rcontrol_s4_ts_ms_hi')|int(0) * 65536) +
(states('sensor.rcontrol_s4_ts_ms_lo')|int(0)) }}
    icon: mdi:clock-outline

- switch:
  # "Synced" Switches: zeigen den echten Istzustand, schalten aber über die
  Coil-Switches.
  # Hinweis: Wenn du die `name:` Werte änderst, ändern sich auch die Entity
  IDs.
  # Dann müssen die unten referenzierten `entity_id:` entsprechend angepasst
  werden.
- name: "rcontrol_relay1"
  state: "{{ is_state('binary_sensor.rcontrol_relay1_state', 'on') }}"
  turn_on:
    - service: switch.turn_on
      target:
        entity_id: switch.rcontrol_relay1_coil
  turn_off:
    - service: switch.turn_off
      target:
        entity_id: switch.rcontrol_relay1_coil

- name: "rcontrol_relay2"
  state: "{{ is_state('binary_sensor.rcontrol_relay2_state', 'on') }}"
  turn_on:
    - service: switch.turn_on
      target:
        entity_id: switch.rcontrol_relay2_coil
  turn_off:
    - service: switch.turn_off
      target:

```

```

        entity_id: switch.rcontrol_relay2_coil

- name: "rcontrol_relay3"
  state: "{{ is_state('binary_sensor.rcontrol_relay3_state', 'on') }}"
  turn_on:
    - service: switch.turn_on
      target:
        entity_id: switch.rcontrol_relay3_coil
  turn_off:
    - service: switch.turn_off
      target:
        entity_id: switch.rcontrol_relay3_coil

- name: "rcontrol_relay4"
  state: "{{ is_state('binary_sensor.rcontrol_relay4_state', 'on') }}"
  turn_on:
    - service: switch.turn_on
      target:
        entity_id: switch.rcontrol_relay4_coil
  turn_off:
    - service: switch.turn_off
      target:
        entity_id: switch.rcontrol_relay4_coil

```

Home Assistant: Relais-Zustand korrekt anzeigen (bei lokaler Änderung)

Wenn ein Relais **lokal am Gerät** (Taster/GUI/Automation) geschaltet wird, möchtest du in Home Assistant den **Istzustand** sehen.

Diese Firmware stellt den Istzustand redundant bereit:

- **Discrete Inputs Bits 4..7:** Relay State (empfohlen für HA `binary_sensor`)
- **Input Registers 60..63:** Relay State als 0/1

Empfehlung:

- Verwende im UI die **Template-Switches (syncd)** (`switch.rcontrol_relay1..4`).
 - Anzeige: echter Istzustand aus `binary_sensor.rcontrol_relayX_state`
 - Schalten: über `switch.rcontrol_relayX_coil` (Modbus Coil)

Damit bleibt Home Assistant korrekt synchron, selbst wenn sich der Zustand lokal am Gerät ändert.

Optional:

- Für S2..S4 entsprechend die Offsets verwenden:
 - Event Counter: $30 + 2*i$ (lo) / $30 + 2*i + 1$ (hi)
 - Timestamp: $40 + 2*i$ (lo) / $40 + 2*i + 1$ (hi)